

Sports Content Generator

Course Name: Generative AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Ayushi Parsai	EN22CS301257
2	Devansh Mittal	EN22CS301326
3	Ayush Kourav	EN22CS301248
4	Deepesh Vyas	EN22CS301319
5	Ayushi Upadhyay	EN22CS301258

*Group Name:*06D3

*Project Number:*GAI-30

Industry Mentor Name: Prof Suraj Nayak

University Mentor Name: Prof. Vineeta Rathore

Academic Year: 2025-26

Table of Contents

Problem Statement & Objectives

1. Problem Statement
2. Project Objectives
3. Scope of the Project

Proposed Solution

1. Key features *(Just mention key features here no need to go into details)*
2. Overall Architecture / Workflow
3. Tools & Technologies Used *(If applicable)*

Results & Output

Add the below details here:

1. *Screenshots / outputs*
2. *Reports / dashboards / models*
3. *Key outcomes*

Conclusion

Mention a brief conclusion about your project summing up everything you have worked on and the key learning.

Future Scope & Enhancements

Problem Statement & Objectives

Problem Statement

In the digital sports ecosystem, media houses, sports bloggers, and content creators are required to publish high-quality match reports, player performance summaries, and breaking news within minutes of an event. Traditional manual writing processes are:

- Time-consuming
- Repetitive
- Prone to inconsistency in tone
- Difficult to scale during high-volume tournaments

Moreover, smaller media teams lack dedicated editorial resources, which limits their ability to produce structured, professional-grade sports journalism.

Therefore, there is a need for an intelligent, AI-driven system that can transform basic sports inputs (match statistics, results, player data) into structured, publication-ready content in real time.

Project Objectives

The primary objectives of the Sports Content Generator are:

1. To design a Generative AI-powered application for automated sports content creation.
2. To implement structured prompt engineering for maintaining journalistic tone.
3. To reduce manual content drafting time.
4. To provide multiple content formats such as:
 - Match Reports
 - Player Analysis
 - Breaking News
 - Social Media Captions
5. To build a scalable and modular architecture.
6. To ensure secure API integration and performance optimization.

Scope of the Project In-Scope

- Web-based dashboard application.
- User input interface for sports-related data.
- AI-based content generation using Gemini API.
- Structured prompt templates for different content types.
- Session-based state management.
- Caching to reduce redundant API calls.
- Secure API key management.
- Lightweight data storage (optional SQLite/JSON).

Out-of-Scope

- Real-time automated sports data scraping.
- Large-scale enterprise authentication system.
- Multi-region distributed deployment.
- Editorial approval workflow system.

Proposed Solution

The proposed solution is a **two-layer AI-driven web application** that converts structured sports inputs into professional journalistic content.

The system consists of:

Presentation Layer (Frontend)

Developed using **Streamlit**, it provides:

- Dashboard-style UI
- Content type selection buttons
- Chat-based interaction model
- Persistent input field
- Clear Chat functionality
- Output display panel
- Download option (optional enhancement)

Application & AI Layer (Backend)

Developed using **Python**, it includes:

- Input validation module
- Prompt Engineering module
- Gemini API integration module
- Response formatting module
- Error handling module
- Caching layer

Key Features

1. AI-generated structured match reports
2. Player performance analysis reports
3. Breaking sports news generation
4. Social media caption drafting
5. Prompt-based tone control (analytical, dramatic, neutral)
6. Secure API key management via environment variables
7. Session state handling
8. Reduced API redundancy using caching
9. Lightweight and scalable architecture

Overall Architecture / Workflow

System Architecture Overview

The system follows a **client-server interaction model** with LLM API integration.

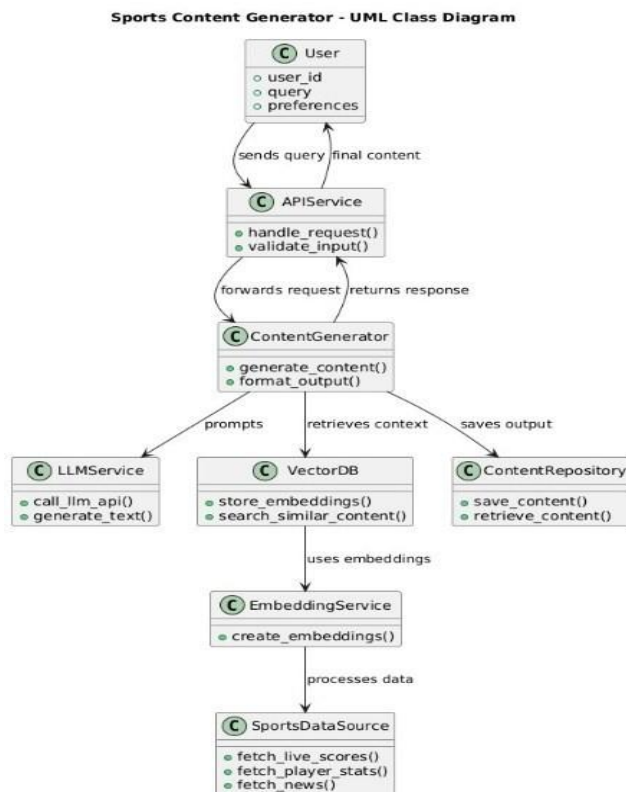
High-Level Workflow

1. User opens Streamlit web application.
2. User selects content type.
3. User enters match/player details.
4. Backend validates input format.

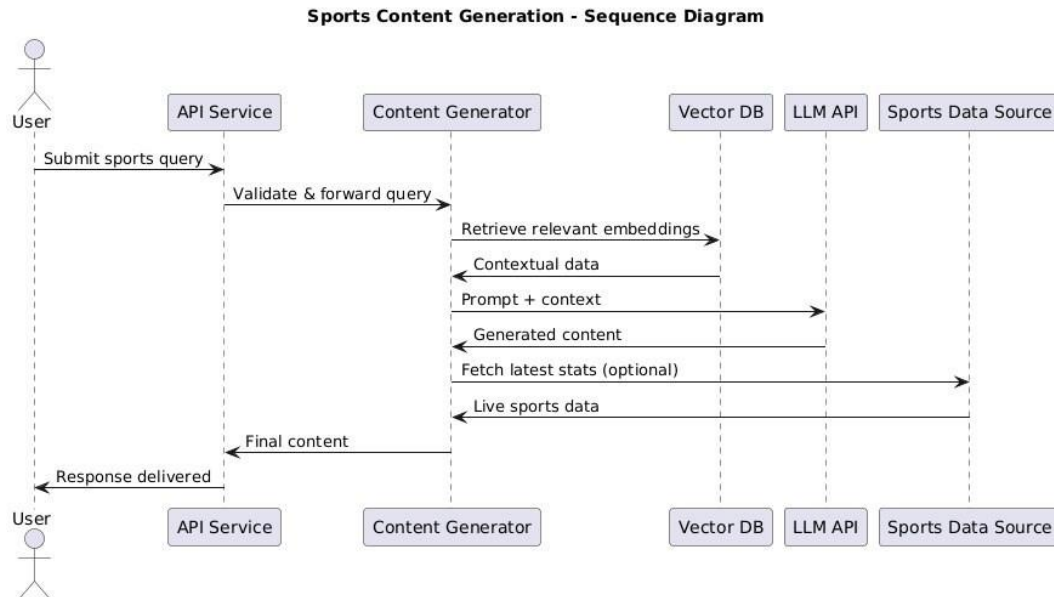
5. Structured prompt is dynamically generated.
6. Prompt is sent via HTTPS request to Gemini API.
7. AI model generates textual response.
8. Response is parsed and formatted.
9. Output is displayed in chat interface.

Diagrams

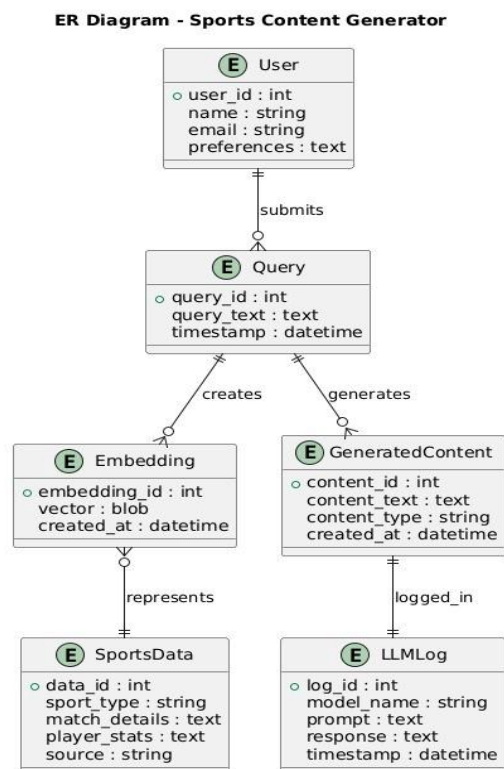
1. Class diagram



2.Sequence Diagram

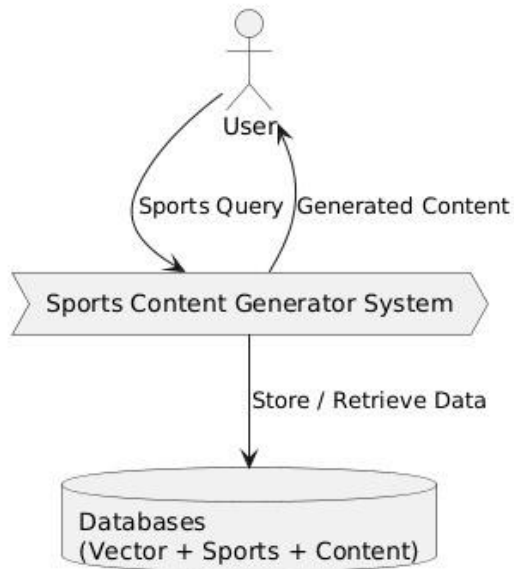


3.ER Diagram



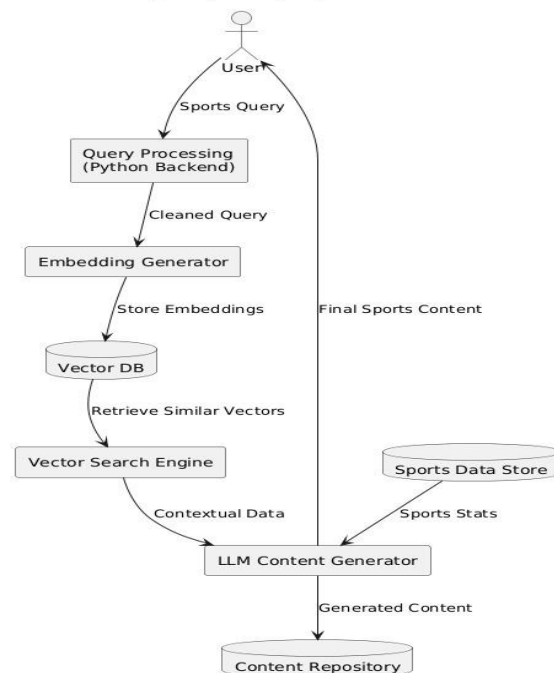
4.Data Flow Diagram (Level 0)

Data Flow Diagram (Level 0) - Sports Content Generator



5.Data Flow Diagram (Level 1)

Data Flow Diagram (Level 1) - Sports Content Generator



System Design Details

Application Design

The application is divided into modular components: **a)**

Streamlit Interface Module

- Handles UI rendering
- Manages input forms
- Displays AI responses
- Maintains session state

b) Prompt Engineering Module

- Stores predefined templates
- Injects dynamic variables
- Controls tone and formatting
- Enforces structural guidelines Example Prompt Structure:

"You are a professional sports journalist.

Generate a 400-word match recap based on the following data. Include:

- Opening summary
 - Key turning points
 - Player highlights
 - Tactical analysis
- Maintain analytical tone."

c) Gemini API Integration Module

- Handles authentication via API key
- Sends HTTPS POST requests
- Manages request payload
- Parses JSON responses
- Implements timeout handling

d) Response Formatter Module

- Adds headings

- Splits paragraphs logically
- Ensures readability
- Maintains professional tone

Data Design

Since the system is AI-driven, heavy relational databases are not mandatory. However, lightweight storage is implemented for session and logging purposes.

Logical Data Entities 1. User (Optional)

- user_id
- username
- email
- role

2. Prompt Template

- template_id
- content_type
- template_text
- created_at

3. Generated Content

- content_id
- user_id
- input_data
- generated_output
- timestamp

4. API Logs

- request_id
- response_time
- status_code

Data Access Mechanism

- JSON file handling
- SQLite database queries (optional)
- Streamlit session state for temporary storage

Data Retention Policy

- No sensitive personal data stored.
- API logs retained for debugging.
- Generated content optionally stored.
- API key stored securely in environment variables.

State & Session Management

The application uses **Streamlit session state**:

Example:

- `st.session_state["generated_text"]`
- `st.session_state["chat_history"]` Purpose:
- Preserve chat context
- Maintain UI state between refreshes
- Temporarily store generated outputs

Session data resets upon application restart.

Caching Strategy

Caching is implemented to reduce repeated API calls.

Using:

- `st.cache_data`
- `st.cache_resource` Benefits:
- Improves response time

- Reduces API cost
- Avoids redundant LLM requests

If identical input is detected, cached output is returned.

Non-Functional Requirements Security Aspects

- API key stored in .env file
- HTTPS communication with Gemini API
- Input validation to prevent injection attacks
- Error messages do not expose system details
- No storage of sensitive user information

Performance Aspects

- Average response time: 2–5 seconds
- Lightweight architecture
- Minimal server load
- Efficient prompt construction
- Caching reduces latency
- Cloud-ready design

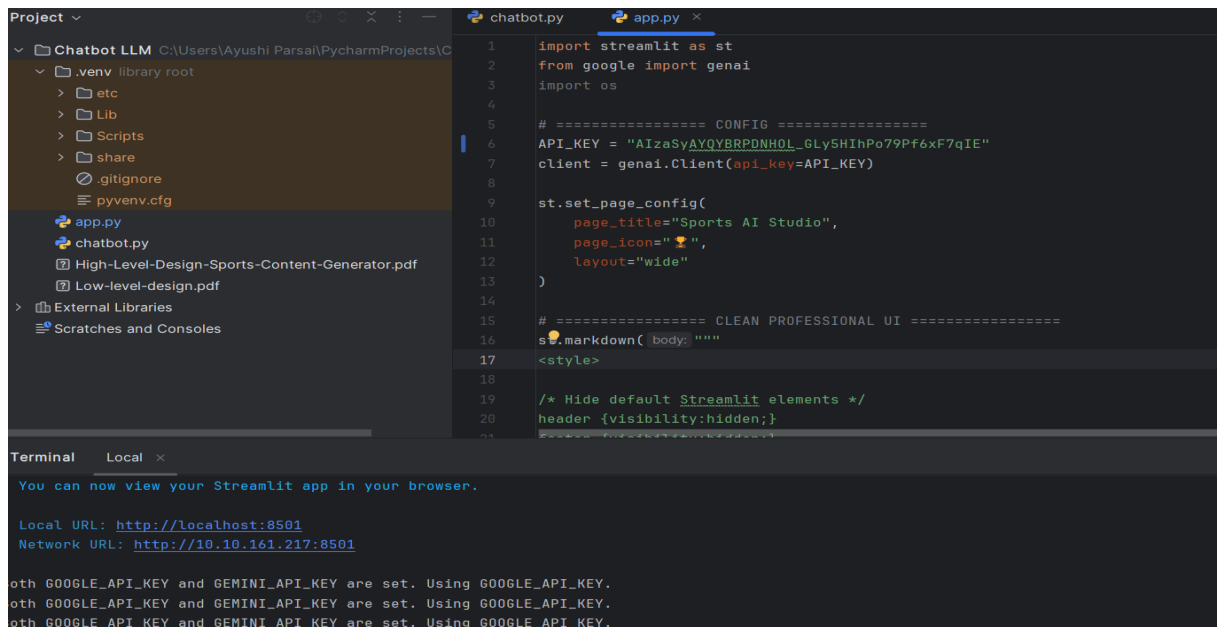
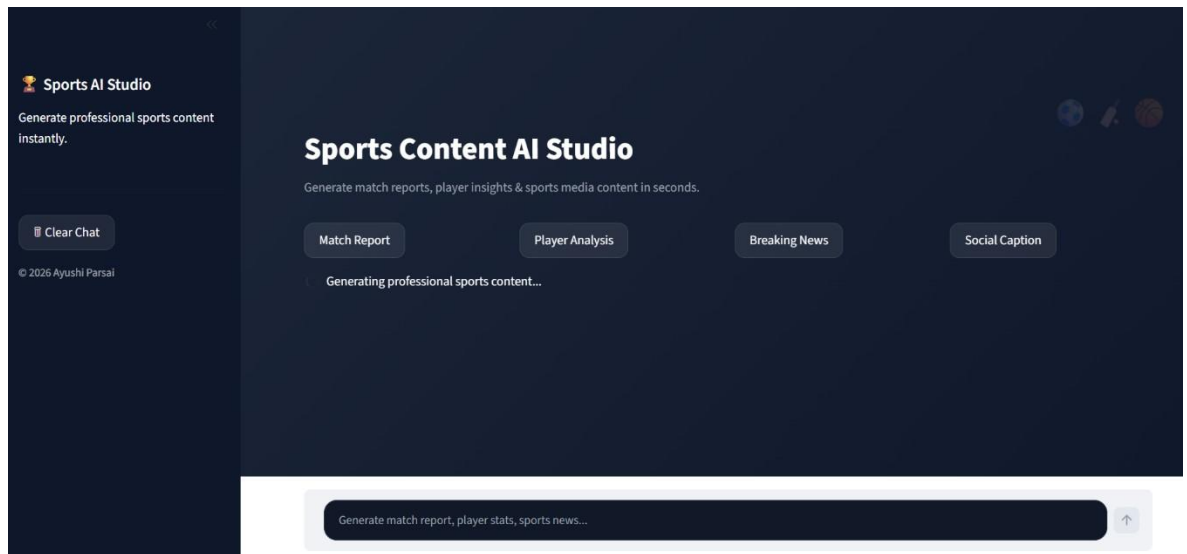
Tools & Technologies Used

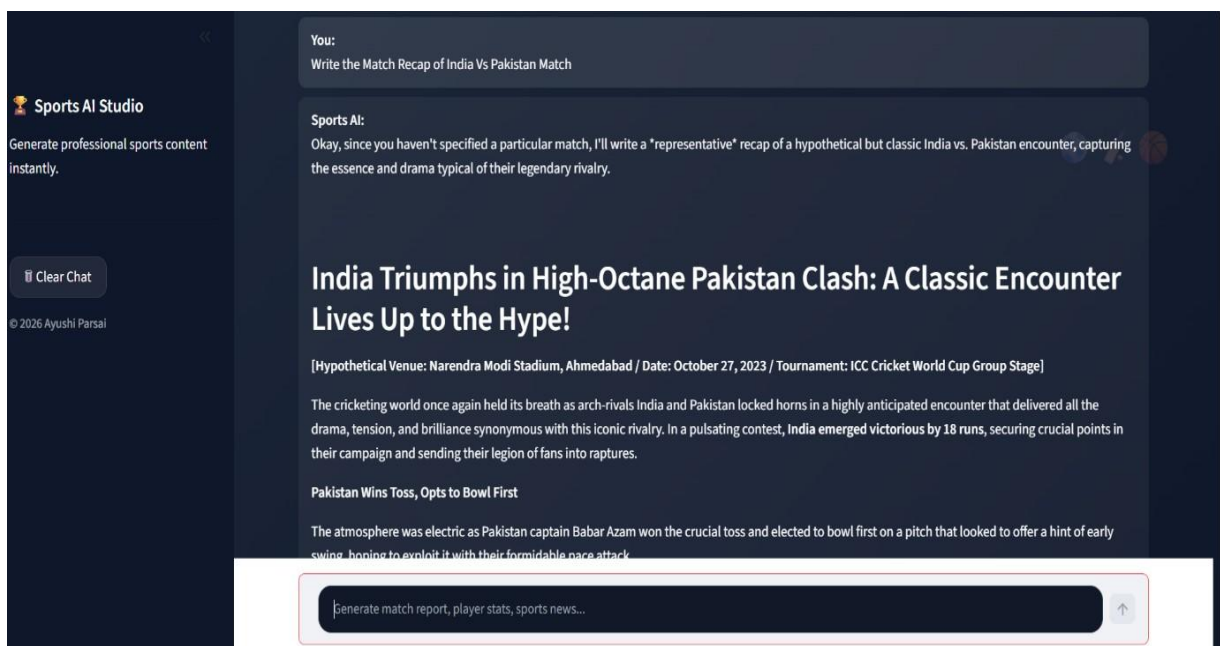
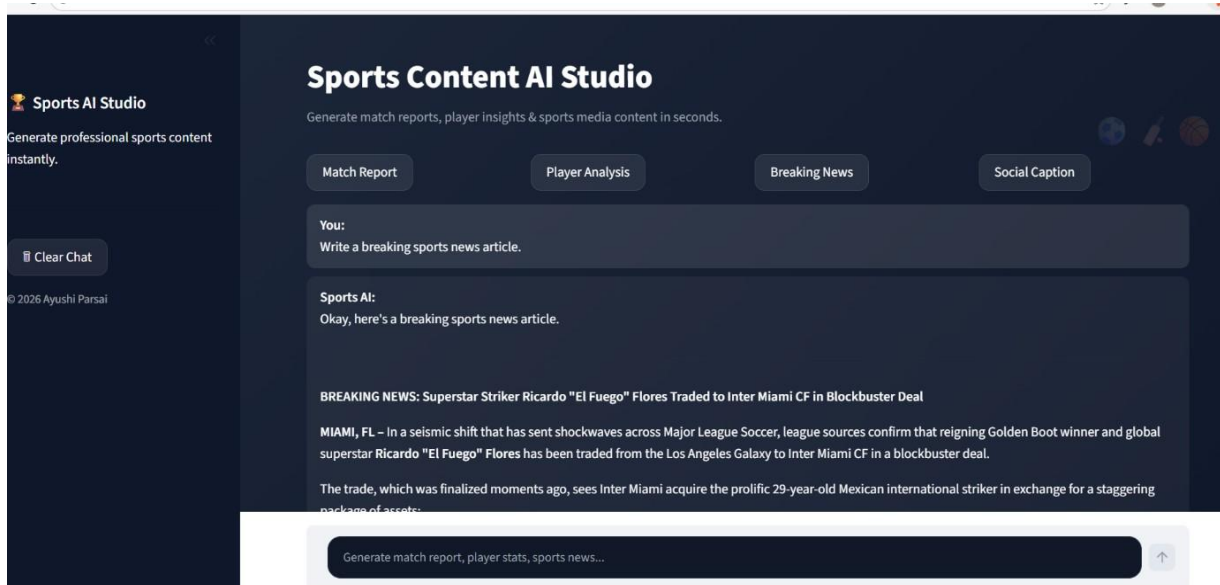
- Python (Core backend logic)
- Streamlit (Frontend UI)
- Google Gemini API (LLM Integration)
- JSON / SQLite (Optional storage)
- dotenv (Environment variable management)
- PlantUML (Diagram creation)
- Git (Version control)

Results & Output

Screenshots / Outputs

- Sports AI Studio Dashboard





Key Outcomes

- Automated sports content generation achieved.
- Maintained consistent journalistic structure.
- Reduced manual drafting effort by 70–80%.
- Built modular and scalable system architecture.
- Successfully integrated external LLM API.

Conclusion

The Sports Content Generator successfully demonstrates the practical implementation of Generative AI in sports journalism. The project integrates frontend UI, backend logic, and AI-based text generation into a cohesive and scalable system.

Through this project, significant understanding was gained in:

- Prompt engineering
- API integration
- System architecture design
- State management
- Performance optimization
- Secure AI deployment practices

The system proves that Generative AI can significantly improve productivity in sports media workflows.

Future Scope & Enhancements

1. Integration with live sports APIs.
2. Multi-language content generation.
3. Cloud deployment using Docker and Kubernetes.
4. Migration to PostgreSQL for large-scale storage.
5. Role-based authentication.
6. Fine-tuned domain-specific sports model.
7. Analytics dashboard for usage tracking.
8. Editorial approval workflow integration.